1D Arrays in C

Definition of 1D Array

A 1D array is a collection of elements of the same data type (homogenous) stored in contiguous memory locations. It is a linear data structure where elements are accessed using indices.

Need for 1D Arrays

1. Efficiently store and manage a large number of elements of the same type.
2. Reduce redundancy by avoiding multiple variable declarations.
3. Enable easy iteration and manipulation using loops.
4. Essential for storing sequences like marks, sales, or sensor readings.

Declaration of 1D Array

c

```
data_type array_name[size];
```

- **data_type**: Type of elements (e.g., int, float, char).
- **array_name**: Name of the array.
- **size**: Number of elements.

Example:

c

```
int numbers[5]; // Array to store 5 integers
```

Types of Initialization

1. **Compile-Time Initialization**:
    - Values are assigned during array declaration.

```
int numbers[5] = {10, 20, 30, 40, 50};
```

2. **Run-Time Initialization**: Values are assigned during program execution using loops or user input.

```
int numbers[5];
for (int i = 0; i < 5; i++) {
   scanf("%d", &numbers[i]);
}
```

3.

Memory Representation

- Each element is stored in contiguous memory locations.

Formula for accessing the address of the i-th element:

```
Address of array[i] = Base address + (i * size_of_data_type)
```

-

Accessing Elements

Elements can be accessed using their index.

c

```
printf("%d", numbers[2]); // Access 3rd element
```

Basic Programs Using 1D Array

Program 1: Find Maximum Element

c

```
#include <stdio.h>
int main() {
   int arr[5], max;
```

```
    printf("Enter 5 numbers: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    max = arr[0];
    for (int i = 1; i < 5; i++) {
        if (arr[i] > max)
            max = arr[i];
    }

    printf("Maximum Element: %d\n", max);
    return 0;
}
```

Program 2: Addition of Elements

c

```
#include <stdio.h>
int main() {
    int arr[5], sum = 0;

    printf("Enter 5 numbers: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
        sum += arr[i];
    }

    printf("Sum of Elements: %d\n", sum);
    return 0;
}
```

2D Arrays in C

Definition of 2D Array

A 2D array is a collection of elements organized in rows and columns. It is often referred to as a matrix.

Need for 2D Arrays

1. Efficiently handle data in tabular format, like marksheets or images.
2. Useful for matrix operations (e.g., addition, multiplication).
3. Manage multiple sets of related data.

Declaration of 2D Array

```
data_type array_name[rows][columns];
```

- **data_type**: Type of elements.
- **rows**: Number of rows.
- **columns**: Number of columns.

Example:

```
int matrix[3][4]; // 3 rows and 4 columns
```

Types of Initialization

Compile-Time Initialization:

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Run-Time Initialization:

```
int matrix[2][3];

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        scanf("%d", &matrix[i][j]);
    }
}
```

Memory Representation

- Stored row by row in memory.

Formula for accessing the address of an element:

Address of matrix[i][j] = Base address + (i * columns + j) * size_of_data_type

---

Accessing Elements

Access using indices:

```
printf("%d", matrix[1][2]); // Element at 2nd row, 3rd column
```

Basic Programs Using 2D Array

Program 1: Matrix Addition

```
#include <stdio.h>
int main() {
   int a[2][2], b[2][2], sum[2][2];

   printf("Enter elements of matrix A: ");
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         scanf("%d", &a[i][j]);
      }
   }

   printf("Enter elements of matrix B: ");
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         scanf("%d", &b[i][j]);
      }
   }

   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         sum[i][j] = a[i][j] + b[i][j];
      }
```

```
    }

    printf("Resultant Matrix:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Program 2: Matrix Multiplication

```
#include <stdio.h>
int main() {
    int a[2][2], b[2][2], product[2][2] = {0};

    printf("Enter elements of matrix A: ");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Enter elements of matrix B: ");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            scanf("%d", &b[i][j]);
        }
    }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                product[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    printf("Resultant Matrix:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
```

```
            printf("%d ", product[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Program 3: Transpose of a Matrix

```
#include <stdio.h>
int main() {
    int matrix[2][3], transpose[3][2];

    printf("Enter elements of 2x3 matrix: ");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    printf("Transpose:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Video link to practise programs
https://www.youtube.com/watch?v=sEiMDFdbPGo

Strings in C

Definition of Strings

In C, a **string** is a sequence of characters terminated by a special null character (\0). It is essentially an array of characters.

Need for Strings

1. **Text Manipulation**: Strings are used to handle and manipulate text, such as names, messages, or file contents.
2. **Ease of Representation**: Simplifies working with sequences of characters.
3. **Flexibility**: Strings can store sentences, process input, and perform operations like concatenation, comparison, and case conversion.

Declaration of Strings

Strings are declared as character arrays:

```
char str[size];
```

Example:

```
char name[20];  // Can store a string of up to 19 characters (+1 for '\0')
```

Types of Strings

1. **Character by Character Input**: Enter each character individually.
2. **Direct Sentence Input**: Input and store the entire sentence at once.

Initialization of Strings

Character-by-Character Initialization

c

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Direct Sentence Initialization

c

```
char str[6] = "Hello";  // Automatically adds '\0'
```

Why Do We Omit & in scanf for Strings?

- In scanf("%s", str);, the name of the string (e.g., str) is itself a pointer to the first character.
- No need for & because the string already represents an address.

Programs Using Strings

1. Concatenation of Strings

Using string.h:

```
#include <stdio.h>
#include <string.h>
int main() {
   char str1[20] = "Hello, ";
   char str2[20] = "World!";
   strcat(str1, str2);  // Concatenates str2 to str1
   printf("Concatenated String: %s\n", str1);
   return 0;
}
```

Without string.h:

```
#include <stdio.h>
int main() {
   char str1[20] = "Hello, ";
   char str2[20] = "World!";
   int i = 0, j = 0;

   // Find the end of str1
   while (str1[i] != '\0') i++;
   // Append str2 to str1
```

```
    while (str2[j] != '\0') {
        str1[i] = str2[j];
        i++;
        j++;
    }
    str1[i] = '\0';

    printf("Concatenated String: %s\n", str1);
    return 0;
}
```

---

2. Finding the Length of a String

Using string.h:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[20] = "Hello";
    printf("Length of the String: %lu\n", strlen(str));
    return 0;
}
```

Without string.h:

```
#include <stdio.h>
int main() {
    char str[20] = "Hello";
    int length = 0;

    while (str[length] != '\0') {
        length++;
    }

    printf("Length of the String: %d\n", length);
    return 0;
}
```

---

3. Changing Case of Characters

Using string.h: Not directly possible. Must write a custom implementation.

Without string.h:

c

```c
#include <stdio.h>
int main() {
    char str[20] = "Hello";
    int i = 0;

    printf("Original String: %s\n", str);

    // Change to Uppercase
    while (str[i] != '\0') {
        if (str[i] >= 'a' && str[i] <= 'z') {
            str[i] = str[i] - 32;  // Convert to uppercase
        }
        i++;
    }

    printf("Uppercase String: %s\n", str);

    // Change to Lowercase
    i = 0;
    while (str[i] != '\0') {
        if (str[i] >= 'A' && str[i] <= 'Z') {
            str[i] = str[i] + 32;  // Convert to lowercase
        }
        i++;
    }

    printf("Lowercase String: %s\n", str);
    return 0;
}
```

Memory Representation of Strings

- **Character Array**: Each character occupies 1 byte in memory.
- **Null Terminator**: The null character (\0) signifies the end of the string.

Example:

For char str[6] = "Hello";, the memory layout is:

Index	0	1	2	3	4	5
Value	'H'	'e'	'l'	'l'	'o'	'\0'

Key Takeaways

- Strings are crucial for handling text data in programs.
- Use string.h for convenience, but understand manual implementation for better understanding of string manipulation.
- Understanding memory representation and null termination is vital to avoid errors like buffer overflows.